

Two Julia Implementations of the Trigger/Target Particle Filter: A Comparison

Prepared for the `PartiallyObservedMarkovProcesses.jl` project

September 8, 2026

Abstract

Both repositories implement the same generalized sequential Monte Carlo (particle filter) recursion for a partially observed Markov process: resampling is triggered by a threshold on the effective sample size, and, when it occurs, is tempered by a power β that interpolates between equally weighted resampling and an untouched weighted particle representation. This note compares Aaron King’s Julia translation (`PartiallyObservedMarkovProcesses.jl`, `aarons/src/pfilter.jl`, at commit `1610d34`) against Debsurya’s Julia translation (`PartiallyObservedMarkovProcesses.jl`, `src/pfilter.jl`), which share the same trigger/target interface and the same recursive structure. The principal finding is that power-tempered resampling discards a normalizing constant C whenever $0 < \beta < 1$: dropping it, as Aaron’s Julia translation and the R `pomp` package’s `wpfilter.c` both do, biases the running log likelihood low by $\Theta(1/n)$ per resampling step, a claim established here by an exact two-particle calculation and confirmed by an end-to-end test built specifically to be sensitive to it; Debsurya’s translation restores C . The note also documents an indexing error, present in the compared Aaron King Julia translation, that assigned the retained weight by resampling slot rather than by selected ancestor. Aaron King has since fixed this himself, independently, in commit `1d8dbc0`, in a form equivalent to Debsurya’s, so it is recorded here as a resolved historical point — and the comparison against it explicitly as a comparison against the pre-`1d8dbc0` code — rather than as a live divergence. Two further, still-live differences are documented: an unequally weighted terminal ancestry draw, which Aaron’s code initiates uniformly rather than proportional to the terminal weights, and the presence, in Debsurya’s translation only, of a per-step resampling diagnostic. It closes with the resulting unification of `wpfilter` as a thin wrapper around `pfilter`.

1 The trigger/target resampling scheme

Both implementations run the same filtering recursion. At each observation time, particles $x_1, \dots, x_n$ carry linear weights $w_1, \dots, w_n$, normalized so that $\bar{w} = n^{-1} \sum_j w_j = 1$. The effective sample size

$$\text{ESS} = \frac{\left(\sum_j w_j\right)^2}{\sum_j w_j^2}$$

is compared against a threshold set by the `trigger` parameter, $\text{trigger} \in [0, 1]$: resampling occurs whenever $\text{ESS} \leq \text{trigger} \cdot n$. `trigger = 1` resamples at every observation time (the classical bootstrap filter); `trigger = 0` never resamples, and the incremental weights are simply carried forward, multiplied into the running weight vector at the next observation time.

When resampling does occur, it is tempered by the `target` parameter $\beta \in [0, 1]$: particle j is selected as an ancestor with probability proportional to $w_j^{1-\beta}$, and each selected particle retains, going forward, a weight proportional to w_j^β (the weight of the particular ancestor selected), renormalized to unit mean across the new particle set. This is a genuine one-parameter family: at $\beta = 0$, ancestors are drawn proportional to the full weight w_j and every survivor is reweighted to 1 — the standard bootstrap resampling step. At $\beta = 1$, ancestors are drawn without regard to weight ($w_j^0 = 1$ for all j), and each survivor retains its ancestor’s original weight $w_j^1 = w_j$ — since uniform systematic selection of n ancestors from n slots reproduces the identity permutation up to a vanishing boundary effect, this carries the full weighted particle representation forward essentially unchanged. Both repositories implement this design; the divergences below concern its realization in code, not the design itself.

2 A positional-indexing error in weight retention, since fixed upstream

This section compares against Aaron’s Julia translation as it stood at commit 1610d34, the version installed locally when this comparison was prepared; it is not a live difference between the two repositories today; see the note at the end of the section. The underlying reasoning is kept because it is used again, unchanged, in §3 below.

Once ancestor indices $p(1), \dots, p(n)$ have been drawn from the cumulative sums of $w_j^{1-\beta}$, the retained weight w_j^β must travel *with the particle it came from*: since $w_j = w_j^{1-\beta} \cdot w_j^\beta$ and the first factor has already been consumed by the selection probability, the second factor is what remains to be assigned to the particle now occupying position $k = p^{-1}(j)$ in the resampled array.

Aaron’s Julia `systematic_resample!` instead raises the entire weight vector to the power β *in place, in its original positional order*, after the ancestor indices have already been drawn:

Listing 1: Aaron’s `systematic_resample!` (PartiallyObservedMarkovProcesses.jl/aarons/src/pfilter.jl, lines 336–363)

```
systematic_resample!(
    p::AbstractArray{I,1},
    w::AbstractArray{W,1},
    ucum::AbstractArray{W,1},
    beta::Float64,
) where {I,W} = begin
    s::W = 0
    alpha = 1-beta
    @inbounds for j in eachindex(w)
        s += w[j]^alpha
        ucum[j] = s
    end
    n::I = length(w)
    i::I = 1
    du::W = s/n
    u::W = -du*rand(W)
    @inbounds for j in eachindex(p)
        u += du
        while (u > ucum[i] && i < n)
            i += 1
        end
        p[j] = i
    end
    w .^= beta
    w ./= mean(w)
    nothing
end
```

The particle *states* are copied correctly downstream (`xf .= xp[p]`), so state k correctly becomes the state of ancestor $p(k)$, but the weight vector `w` is exponentiated positionally: the new weight at position k is w_k^β — the β -power of whatever particle *used to occupy* position k — not $w_{p(k)}^\beta$, the β -power of the ancestor actually selected. Whenever the resampling permutation p moves mass between positions, which is the norm rather than the exception, the state and its retained weight come from two different particles.

Aaron’s R implementation of the same routine (the C code behind `wpfilter` in the R `pomp` package) does not have this defect: it copies the retained weight of the selected ancestor directly, so the weight is indexed by the ancestor, not by position:

Listing 2: R `wpfilter.c`, inside the ancestor-selection loop, as reported by Debsurya

```
wt[k] = ws[sp]; /* ws = carried weights, sp = selected ancestor index */
```

Debsurya’s Julia translation takes the ancestor-indexed form, following the R form rather than the positional form above:

Listing 3: Debsurya’s `systematic_resample!` (PartiallyObservedMarkovProcesses.jl/src/pfilter.jl, lines 457–491), retained-weight step

```
@inbounds for j in eachindex(p)
    ucum[j] = w[p[j]]^beta # retained weight of the selected ancestor
end
```

```

end
@inbounds for j in eachindex(w)
    w[j] = ucum[j]
end
m::W = mean(w)
w ./= m
log(m*s/n)

```

Here `w[p[j]]` is evaluated *before* `w` is overwritten, using `ucum` as scratch space, so each new weight is correctly indexed by the selected ancestor. At $\beta = 0$ the two forms coincide trivially (every weight is 1 regardless of index), which is presumably why the error had not surfaced in the classical (target = 0) filter; it is only visible for $\beta > 0$.

This is no longer a difference. Aaron King fixed the positional form himself, independently, in commit `1d8dbc0` (“fix bug in weighted particle filter,” 2026-09-08), which postdates `1610d34` and is not an ancestor of the local clone compared above:

Listing 4: Aaron’s `systematic_resample!` after commit `1d8dbc0`, retained-weight step

```

@inbounds for j      eachindex(p)
    ucum[j] = w[p[j]]
end
@inbounds for j      eachindex(w)
    w[j] = ucum[j]^
end

```

which is equivalent to Debsurya’s form: both index the retained weight by the selected ancestor $p(k)$ rather than by resampling slot k . The finding above is accordingly recorded as a resolved historical point, not a present divergence between the two implementations. It is kept in this note in full because the same reasoning about which quantity must travel with the selected ancestor, rather than with the slot it lands in, is exactly what §3 needs to derive the normalizing constant discarded at partial resampling.

3 The normalizing constant discarded at partial resampling

Write $w_1, \dots, w_n$ for the unit-mean weights immediately prior to resampling, and $S = \sum_j w_j^{1-\beta}$ for the normalizing sum of the selection probabilities $q_j = w_j^{1-\beta}/S$. Because ancestors are drawn from q rather than from the weights themselves, the sampling measure has changed, and the properly weighted representation assigns the particle selected as ancestor A_k at position k the importance weight

$$R_k = \frac{w_{A_k}}{n q_{A_k}} = \frac{S}{n} w_{A_k}^\beta,$$

whose sample mean over the n selections,

$$C = \frac{1}{n} \sum_{k=1}^n R_k = \frac{S}{n} m, \quad m = \frac{1}{n} \sum_{k=1}^n w_{A_k}^\beta,$$

is the resampling step’s own contribution to the normalizing constant of the unnormalized (Feynman–Kac) measure the filter estimates. Both implementations instead store the retained weights renormalized to unit mean, $V_k = R_k/C$ — necessary since every other routine in the filter (the effective-sample-size and conditional-log-likelihood computation in particular) assumes a unit-mean weight vector on entry — and neither multiplies C back into the likelihood accumulator. Aaron’s Julia routine returns `nothing`, its caller discarding it:

Listing 5: Aaron’s `pfilt_step_comps!` (PartiallyObservedMarkovProcesses.jl_aarons/src/pfilter.jl, line 228)

```

systematic_resample!(p, w, work, target)    # return value discarded

```

and R `pomp`’s `wpfilter.c` discards it too, by a different route: it leaves the carried weights unnormalized and computes each conditional log likelihood as a ratio of successive *total* masses (`loglik = maxlogw + log(w) - log(oldw)`, line 110), so that the mass change across a resampling step is simply never entered into the likelihood at all — it becomes part of the baseline the next step’s ratio is taken against, with no correction of any kind at the step where it arose. C is dropped, not restored, in both of Aaron King’s implementations, and, until now, in this repository as well.

Why dropping C is a real bias

C is conditionally mean one: $\mathbb{E}[m \mid w] = \sum_j q_j w_j^\beta = (\sum_j w_j)/S = n/S$, so $\mathbb{E}[C \mid w] = 1$ exactly. It is tempting to conclude from this alone that dropping C costs only variance, since a factor with conditional expectation one, multiplied into an otherwise-unbiased estimator, cannot change that estimator's expectation. That inference needs independence, or at least the absence of the wrong kind of correlation, and C does not have it: C is a function of which ancestors $A_{1:n}$ were actually selected, and those same ancestors go on to generate *every future contribution to the likelihood*. Knowing $\mathbb{E}[C] = 1$ does not give $\mathbb{E}[C \ell] = \mathbb{E}[\ell]$ once C and ℓ are dependent through a shared cause — here, the resampled particle cloud itself. Dropping C biases the likelihood estimate downward by $\Theta(1/n)$ per resampling step. $C \equiv 1$ identically at $\beta = 0$ (where $S = \sum_j w_j = n$ and every retained weight is 1, so $m = 1$) and at $\beta = 1$ (where $S = n$ trivially and, since uniform systematic selection of n ancestors from n slots reproduces the identity permutation, $m = n^{-1} \sum_j w_j = 1$ as well), so the ordinary bootstrap filter is entirely untouched: this is strictly a partially-retained-weight ($0 < \beta < 1$) phenomenon.

An exact counterexample

Take $n = 2$ particles with unit-mean weights $w = (1.8, 0.2)$ and $\beta = 1/2$. Then $q = (0.75, 0.25)$, and two-particle systematic resampling gives ancestry $(1, 1)$ or $(1, 2)$, each with probability exactly $1/2$, carrying $C = 1.2$ and $C = 0.8$ respectively. Suppose the future contribution from the two particles is $g = (1, 0)$; the correctly weighted predictive quantity is $\frac{1}{n} \sum_i w_i g_i = 0.9$. Dropping C gives $\frac{1}{2}(1) + \frac{1}{2}(0.75) = 0.875$; retaining it gives $\frac{1}{2}(1.2 \cdot 1) + \frac{1}{2}(0.8 \cdot 0.75) = 0.9$, exactly. This has been verified numerically to six decimal places over 4,000,000 replicates.

An end-to-end test, and why an earlier check missed this

`test/iid.jl` contains two test sets. The first uses a latent process with no memory: X_t is drawn afresh, independently of X_{t-1} , at every step. There the one-step predictive density is the same for every particle regardless of which ancestors survive resampling, so the tempered resampling step is exactly unbiased whatever it does with C — this model is blind to the effect by construction, at any number of replicates. The second test uses a frozen binary latent state, $X \sim \text{Bernoulli}(1/2)$ with $X_t = X$ for all t , together with measurement potentials $g_1(1) = 1.8$, $g_1(0) = 0.2$ and $g_2(1) = 1.0$, $g_2(0) = 0.0$; the exact likelihood is $Z = \mathbb{E}[g_1(X)g_2(X)] = 0.9$. At $n = 2$ and $\beta = 1/2$, dropping C gives $\mathbb{E}[\hat{Z}] \approx 0.8879$ against $Z = 0.9$ — a 27-standard-error discrepancy over 200,000 replicates — that vanishes when C is restored.

An earlier check, run instead on the linear-Gaussian reduction of the Gompertz model against its closed-form Kalman likelihood over 400,000 paired replicates at $\beta \in \{0.25, 0.5, 0.75\}$ and $n \in \{8, 16, 64\}$, found no significant difference between the two conventions and was taken, wrongly, as evidence that C needed no restoring. That check had no power: the bias is $\Theta(1/n)$ per resampling step and vanishes identically whenever the one-step predictive density is constant across the cloud, which is very nearly true of a smooth, well-mixing latent process such as log-Gompertz. The design could resolve differences of roughly 0.02% of $\hat{L}$ against a bias under 0.15% — an underpowered test, not a clean null result. A null Monte Carlo comparison is evidence of nothing unless the test model can exhibit the effect in question and the design has power against it; the frozen-state model above was built specifically to have both properties.

The fix

Debsurya's `systematic_resample!` now returns $\log C$ instead of `nothing`, and the caller accumulates it into the running log likelihood:

Listing 6: `PartiallyObservedMarkovProcesses.jl/src/pfilter.jl`, lines 481–490 and 315

```
stot::W = s      # S = sum_j w_j^(1-beta)
...
m::W = mean(w)
w ./= m # subsequent steps rely on the weights having unit mean
log(m*stot/n)
```

Listing 7: The caller, `pfilt_step_comps!`, line 315

```
logLik[] += systematic_resample!(p, w, work, target)
```

Aaron's Julia translation and R `pomp`'s `wpfilter.c` still drop C , unchanged from the description above.

4 Terminal ancestry draw under unequal terminal weights

Whenever $\beta > 0$, or a step is skipped by `trigger`, the terminal particle cloud carries unequal weights: the filtering distribution’s information sits partly in the weights, not solely in the particle multiplicities. Recovering a single properly weighted draw of the filtered trajectory therefore requires initiating the traced lineage at the terminal time with probability proportional to the terminal weights, then tracing ancestry backward through the recorded permutations.

Aaron’s `trace_ancestry!` initiates the draw uniformly, regardless of the terminal weights:

Listing 8: Aaron’s `trace_ancestry!` (PartiallyObservedMarkovProcesses.jl-aarons/src/pfilter.jl, line 397)

```
j::I = rand(axes(perm,2))      # uniform draw, regardless of terminal weights
```

A marginal comment in that file (`# FIXME: check that the first index in the ancestry is correct`) suggests this was already a point of concern. Unlike the positional-indexing error of §2, this one survives commit `1d8dbc0`: the terminal draw there (line 408) is still the uniform `j = rand(axes(perm,2))`, and the `FIXME` comment (now at line 97) is still unresolved. This is accordingly a live difference between the two implementations, not a historical one.

A minimal counterexample makes the resulting bias concrete. Suppose two terminal particles carry weights $W = (0.9, 0.1)$ — an entirely ordinary outcome once $\beta > 0$ or a step goes unresampled. The correct marginal draw of the terminal state assigns particle 1 probability 0.9 and particle 2 probability 0.1, so that a functional such as the traced trajectory’s terminal value has, e.g., $\mathbb{P}(X_T = x^{(1)}) = 0.9$. Aaron’s uniform draw instead assigns each particle probability 0.5 regardless of W , systematically over-weighting the low-probability particle and under-weighting the high-probability one; the traced trajectory is properly weighted for the filtering distribution only by coincidence, when the terminal weights happen to be equal.

Debsurya’s `trace_ancestry!` initiates the draw proportional to the terminal weights before tracing backward:

Listing 9: Debsurya’s weighted `trace_ancestry!` (PartiallyObservedMarkovProcesses.jl/src/pfilter.jl, lines 518–554), initiation step

```
s::W = 0
for k in eachindex(w)
    v::W = w[k]
    s += v
    work[k] = s
end
u::W = s*rand(LogLik)
jj::I = 1
while (u > work[jj] && jj < n)
    jj += 1
end
j = jj
# ... trace backward from j exactly as before
```

with a fallback to the uniform draw only in the degenerate case where every terminal weight underflows to zero. This routine was exercised in the model-based validation against R `pomp` and the closed-form Gompertz likelihood described elsewhere, and directly addresses the concern flagged in Aaron’s comment.

5 A per-step resampling diagnostic

Debsurya’s `PfilterdPompObject` carries an additional field, `resampled::Array{Bool,1}`, recording at each observation time whether systematic resampling actually occurred there (as opposed to being skipped because ESS stayed above the `trigger` threshold), together with an accessor:

Listing 10: PartiallyObservedMarkovProcesses.jl/src/pfilter.jl, lines 29–35

```
"""
    resampled(object)

‘resampled’ extracts the vector of booleans recording, for each
observation time, whether systematic resampling occurred.
"""
resampled(object::PfilterdPompObject) = object.resampled
```

This is set alongside the ancestry permutation in `pfilt_step_comps!` (`resamp[] = true` when resampling occurs, `false` otherwise) and gives a cheap, per-step diagnostic of weight degeneracy over the course of a filtering run, independent of and complementary to the effective sample size trace. Aaron’s `PfilterdPompObject` has no corresponding field.

6 Unification: `wpfilter` as a thin wrapper

In both repositories, the classical bootstrap filter (`trigger = 1`, `target = 0`) and the general trigger/target filter were, at one point, separately implemented engines: a fast path with no weight-carrying overhead for the classical case, and a general path that threads a carried weight vector `wcarry` through the recursion (folded into the incremental log weights at `compute_ess_logLik!`, and consulted at each resampling step). Debsurya’s `pfilter` still dispatches internally between these two engines on the basis of whether `trigger==1 && target==0`, but both are reached through a single public entry point, and `wpfilter` — the name inherited from the R `pomp` package’s separate weighted-filter routine — is now nothing more than a same-signature call-through to `pfilter`:

Listing 11: `PartiallyObservedMarkovProcesses.jl/src/wpfilter.jl`, in full

```
wpfilter(
    object::ValidPompData;
    trigger::Real = 1,
    target::Real = 0,
    kwargs...,
) = pfilter(object; trigger, target, kwargs...)

wpfilter(
    object::PfilterdPompObject;
    Np::Integer = object.Np,
    trigger::Real = object.trigger,
    target::Real = object.target,
    kwargs...,
) = pfilter(object; Np, trigger, target, kwargs...)

wpfilter(...) = error("Incorrect call to 'wpfilter'.")
```

There is exactly one particle-filtering implementation to validate, maintain, and reason about; `wpfilter` survives purely as an API-compatible alias, with defaults (`trigger = 1`, `target = 0`) that in fact route it to the same classical engine as a bare call to `pfilter`. This unification is also what makes it possible for `mif2`, the iterated-filtering routine, to run its inner filtering step under `target > 0`: since there is only one filtering routine underneath both names, a weighted filtering step is automatically available wherever a classical one was, which is not presently true of the R implementation, whose weighted filter does not resample parameters.

7 Summary

Issue	Aaron’s Julia	Debsurya’s Julia
Retained weight indexing (§2)	ancestor-indexed (<i>positional at 1610d34; fixed independently at 1d8dbc0</i>)	ancestor-indexed: $w_{p(k)}^\beta$ (<i>no longer a difference</i>)
Normalizing constant C at $0 < \beta < 1$ (§3)	dropped (<i>bug: biases $\hat{L}$ low by $\Theta(1/n)$ per resampling step</i>)	restored: <code>logLik[] += log C</code>
Terminal ancestry draw (§4)	uniform (<i>unchanged at 1d8dbc0</i>)	proportional to terminal weight
Per-step resampling flag (§5)	absent	<code>resampled(object)</code>
<code>wpfilter</code> (§6)	separate historical engine	thin wrapper over <code>pfilter</code>

Of the five rows, only the third and fourth are differences of the comparison sort this note set out to find: for `target > 0`, the terminal ancestry draw changes the distribution represented by the stored filtered trajectory, and the resampling flag is simply present in one implementation and not the other. Neither disturbs the marginal log-likelihood, so neither would have been caught by comparing likelihood estimates; they were found by reading

the code. The first row was of the same kind — it would have changed which particle’s information survives resampling for target > 0 — but Aaron King fixed it independently before this note was finished, so it is recorded as resolved rather than open. The second row is different in kind from all the others: it is not a difference of code structure but a shared numerical defect, present in Aaron’s Julia translation and in R `pomp` alike, that this repository has now corrected. Unlike the first and third rows, it *does* disturb the marginal log-likelihood, by $\Theta(1/n)$ per resampling step at every $0 < \beta < 1$; the reason it went undetected for as long as it did is that the model used to check the filter’s likelihood against an exact reference happened to be almost exactly the case, described in §3, in which the effect vanishes.